

Streaming Responses

LESSON 04 · MODULE 1 — FOUNDATIONS

What you'll learn

- How `stream=True` flips `agent.run()` from a single response into an iterator of chunks
- The standard four-line loop to print streamed tokens as they arrive
- Why you skip the final `done` chunk (and what would happen if you didn't)
- Which fields on a chunk you actually use, and which to know exist
- One silent fallback to watch for when a Responsible AI policy is attached

Key concepts

`stream=True` changes the return type. Without it, `agent.run()` returns a response object. With it, you get an iterator. Each pass of the loop is one chunk arriving from the model.

The standard streaming loop is four lines. Loop the iterator. First thing in the body: check `chunk.done` and break. Otherwise print `chunk.content` with `end=""` and `flush=True` so Python doesn't buffer — the whole point is to see the text as it arrives.

TAKEAWAY

The standard streaming loop is four lines.

****Why break *before* printing on the done chunk.**** The final chunk in this SDK carries the full accumulated text in `.content`, not just the last token. Print it and you get your whole answer twice. Every prior chunk already emitted its piece, so skipping the done chunk is the right move.

The fields that matter. `chunk.content` (the new tokens for this chunk) and `chunk.done` (terminal marker) handle 90% of streaming code. `chunk.structured_data` is populated on the done chunk when a `response_model` is set (next video). `chunk.artifact_files` shows up when the agent generates files (image generation video).

TAKEAWAY

The fields that matter.

Streaming silently turns off when you attach a Responsible AI policy. Guardrails for PII, toxicity, prompt injection — any of that — and the SDK falls back to non-streaming so the guardrail can scan the full response before any of it reaches the user. Correct behavior for safety; streaming half an answer past a policy check defeats the purpose. But it's *silent*. Your loop still runs, your code still works, you just get one big chunk at the end. If you're building something user-facing with guardrails, plan for it — show a spinner on policy-enabled agents, or render the final chunk as if it were streamed.

TAKEAWAY

Streaming silently turns off when you attach a Responsible AI policy.

Code shown in this lesson

PYTHON

```
# Standard non-streaming call (baseline)
response = agent.run("Write a short paragraph about why streams feel fast.")
print(response.response)
```

PYTHON

```
# Streaming – same agent, one extra keyword
for chunk in agent.run("Write a short paragraph about why streams feel fast.", stream=True):
    if chunk.done:
        break
    print(chunk.content, end="", flush=True)
```

PYTHON

```
# Inspecting every field on every chunk (debug helper)
for chunk in agent.run("Count to three.", stream=True):
    print({
        "content":      chunk.content,
        "delta":        chunk.delta,
        "done":         chunk.done,
        "structured_data": getattr(chunk, "structured_data", None),
        "artifact_files": getattr(chunk, "artifact_files", None),
    })
```

TRY THIS

- 1 Wrap the streaming loop with `time.time()` markers — one before the first chunk arrives, one at the `done` chunk. Run the same prompt non-streaming. Compare *time-to-first-token* vs *total time*. You'll feel the UX difference even when total time is identical.
- 2 Inside the loop, print `chunk.delta` alongside `chunk.content` for every chunk. Watch how the two fields differ (or don't) across the full response. Then attach a Responsible AI policy to the agent and run it again — note what happens to the chunk stream.